

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Оценка времени выполнения программ»

Выполнил: ст. гр. 25BBB1

Какулин В.С.

Самсонов М.А.

Проверил: к.т.н. доцент

Юрова О.В.

к.т.н., ст. преподаватель

Деев М.В.

Пенза 2026

Название:

Оценка времени выполнения программ

Цель работы:

Определение порядка вычислительной сложности и исследование времени выполнения алгоритма перемножения матриц для различных размерностей с построением и сравнением графиков с теоретической оценкой, а также сравнительный анализ эффективности сортировки Шелла, быстрой сортировки и стандартной функции `qsort` на различных типах структурированных и случайных наборов данных.

Лабораторное задание:

Задание 1:

- 1) Вычислить порядок сложности программы (O-символику).
- 2) Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000
- 3) Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2 :

- 1) Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
- 2) Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
- 3) Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
- 4) Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
- 5) Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Ход работы:

Задание 1:

1. Описание используемого алгоритма:

Алгоритм перемножения матриц: осуществляется с помощью трех вложенных циклов. Внешний цикл перебирает строки первой матрицы, средний – столбцы второй матрицы, а внутренний цикл вычисляет скалярное произведение строки и столбца. Теоретическая сложность данного алгоритма составляет $O(N^3)$, как раз за счет вложенности трех циклов for. Это означает, что при увеличении размера матрицы в 2 раза время выполнения должно возрасть примерно в $2^3 = 8$ раз.

2. Описание используемых функций:

clock() – функция, возвращающая процессорное время, затраченное на выполнение программы. Используется для точного замера времени выполнения участков кода.

CLOCKS_PER_SEC – константа, определяющая количество тактов процессора в одной секунде, необходимая для перевода тактов в секунды.

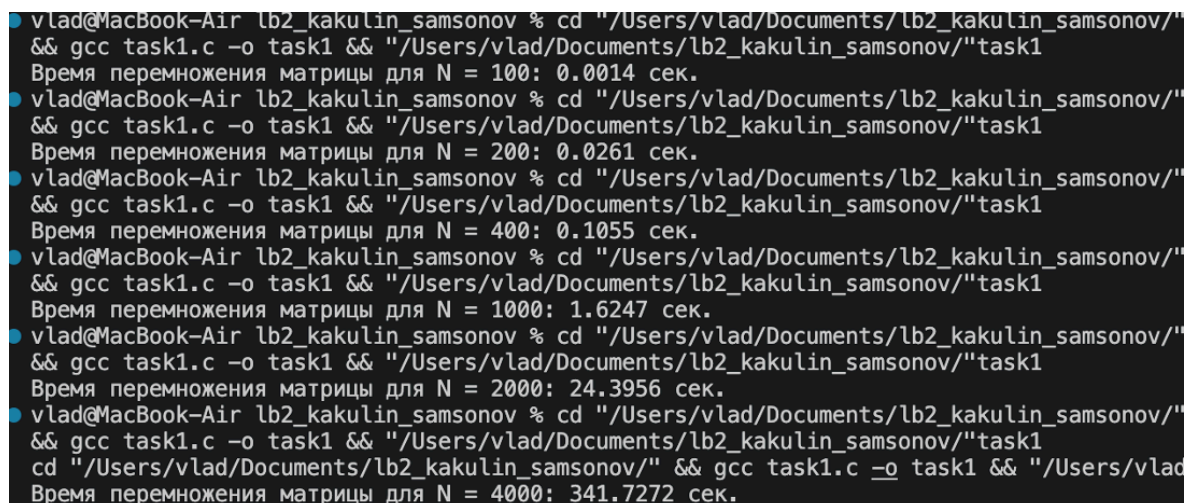
malloc() – функция динамического выделения памяти для создания двумерных массивов произвольного размера.

free() – функция освобождения динамической памяти для предотвращения утечек.

rand() и srand() – функции генерации и инициализации псевдослучайных чисел для заполнения матриц.

time() – функция системного времени, передаваемая в srand() для уникальности последовательности случайных чисел.

Результат выполнения программы:



```
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
&& gcc task1.c -o task1 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task1
Время перемножения матрицы для N = 100: 0.0014 сек.
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
&& gcc task1.c -o task1 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task1
Время перемножения матрицы для N = 200: 0.0261 сек.
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
&& gcc task1.c -o task1 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task1
Время перемножения матрицы для N = 400: 0.1055 сек.
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
&& gcc task1.c -o task1 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task1
Время перемножения матрицы для N = 1000: 1.6247 сек.
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
&& gcc task1.c -o task1 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task1
Время перемножения матрицы для N = 2000: 24.3956 сек.
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
&& gcc task1.c -o task1 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task1
cd "/Users/vlad/Documents/lb2_kakulin_samsonov/" && gcc task1.c -o task1 && "/Users/vlad
Время перемножения матрицы для N = 4000: 341.7272 сек.
```

Рисунок 1 – Результат выполнения программы для разных размеров матриц задания 1.



Рисунок 2 - График зависимости времени выполнения программы от размера матриц задания 1.

Сравнение с теоретической оценкой:

Теоретическая оценка предполагает кубическую зависимость $O(N^3)$, что при удвоении размера матрицы должно увеличивать время выполнения примерно в $2^3 = 8$ раз. Проверим это на практических данных:

При увеличении размера матрицы от 1000 до 2000 (2 раза) время увеличилось с 1,62 с до 24,40 с, то есть примерно в 15 раз.

При увеличении размера от 2000 до 4000 (2 раза) время возросло с 24,40 с до 341,75 с, то есть примерно в 14 раз.

Как видно из расчетов, практические результаты заметно расходятся с чистой теоретической моделью (реальный множитель роста составляет около 14–15 вместо идеальных 8).

Тем не менее, общий характер изменения значений демонстрирует стремительный рост, соответствующий кубической зависимости.

Примечание по размеру $N = 10000$: при попытке запуска программы для $N = 10000$ программа не завершается за разумное время (или требует колоссальных ресурсов). Это обусловлено тем, что для трех матриц размером 10000×10000 требуется около большое кол-во оперативной памяти только под хранение данных, а число операций умножения и сложения достигает порядка триллиона, что делает выполнение стандартного кубического алгоритма на обычном ПК практически невозможным за адекватное время.

Задание 2:

1. Описание используемых алгоритмов:

Сортировка Шелла: Алгоритм усовершенствованной сортировки вставками, основанный на сравнении элементов, находящихся на определенном

расстоянии (шаге) друг от друга. В реализации используется фиксированная последовательность шагов (9, 5, 3, 2, 1). На начальных этапах большие шаги позволяют быстро переместить элементы на нужные позиции, а финальный проход с шагом 1 завершает упорядочивание.

Быстрая сортировка: Рекурсивный алгоритм типа «разделяй и властвуй». Выбирается опорный элемент (в данной реализации – центральный элемент массива), после чего массив разделяется на две части так, чтобы все элементы меньше опорного оказались слева, а больше – справа. Затем функция рекурсивно вызывается для левой и правой частей.

Стандартная сортировка: Библиотечная функция быстрого сортирования из `<stdlib.h>`, которая использует оптимизированные внутренние механизмы для безопасной и быстрой сортировки массивов любых типов данных с помощью функции сравнения `cmp`.

2. Описание используемых функций:

`shell()` – пользовательская функция, реализующая сортировку Шелла с фиксированным набором шагов для поэтапного перемещения элементов.

`qs()` – рекурсивная функция пользовательской быстрой сортировки, выполняющая разбиение массива относительно опорного элемента.

`cmp()` – функция сравнения двух элементов, используемая стандартной функцией `qsort`. Возвращает разницу между значениями для определения порядка сортировки.

`copy()` – вспомогательная функция для копирования элементов из базового массива в рабочий массив перед каждым новым тестом.

`fill_rnd()` – функция генерации псевдослучайных чисел, заполняющая массив случайными значениями в заданном диапазоне с помощью `rand()`.

`fill_asc()` – функция заполнения массива возрастающей последовательностью чисел от 0 до `SIZE - 1`.

`fill_desc()` – функция заполнения массива убывающей последовательностью чисел.

`fill_half()` – функция заполнения массива комбинированной последовательностью, где первая половина отсортирована по возрастанию, а вторая – по убыванию.

`clock()` – функция измерения процессорного времени, затраченного на выполнение участков кода с сортировкой.

`qsort()` – стандартная функция быстрой сортировки из библиотеки `<stdlib.h>`.

Результат выполнения программы:

```
vlad@MacBook-Air lb2_kakulin_samsonov % cd "/Users/vlad/Documents/lb2_kakulin_samsonov/"
& gcc task2.c -o task2 && "/Users/vlad/Documents/lb2_kakulin_samsonov/"task2
1. Случайный набор:
   Сортировка Шелла: 0.3764 сек.
   Быстрая сортировка: 0.0070 сек.
   Стандартная сортировки: 0.0060 сек.

2. Возрастающая последовательность:
   Сортировка Шелла: 0.0007 сек.
   Быстрая сортировка: 0.0012 сек.
   Стандартная сортировки: 0.0003 сек.

3. Убывающая последовательность:
   Сортировка Шелла: 0.7008 сек.
   Быстрая сортировка: 0.0013 сек.
   Стандартная сортировки: 0.0011 сек.

4. Комбинированная последовательность:
   Сортировка Шелла: 0.3509 сек.
   Быстрая сортировка: 1.6307 сек.
   Стандартная сортировки: 0.0023 сек.
```

Рисунок 3 – результат выполнения программы задания 2.

Сравнение результатов:

На основе полученных данных для массива размером 100_000 элементов можно сделать следующие выводы о производительности алгоритмов:

Стандартная функция: показала абсолютную стабильность и лучшую общую производительность на всех типах наборов данных (время выполнения от 0.0003 до 0.0060 сек). Она эффективно справляется как со случайными, так и со специфическими упорядоченными структурами без деградации скорости.

Пользовательская быстрая сортировка: на случайных (0.0070 сек), возрастающих (0.0012 сек) и убывающих (0.0013 сек) массивах работает быстро. Однако на комбинированном наборе (половина возрастающая, половина убывающая) наблюдается резкий рост по времени до 1.6307 сек.

Сортировка Шелла: демонстрирует сильную зависимость от характера исходных данных: на возрастающей последовательности работает быстро (0.0007 сек), так как элементы практически не требуют перестановки; на убывающей последовательности время возрастает до 0.7008 сек, поскольку обратный порядок требует максимального числа сдвигов элементов во внутренних циклах вставки; на случайных и комбинированных наборах показывает стабильный, но более низкий результат (0.35–0.38 сек) по сравнению с обоими вариантами быстрой сортировки из-за ограничений выбранной сетки фиксированных шагов.

Вывод:

В ходе лабораторной работы были изучены принципы измерения времени выполнения программы, оценки вычислительной сложности и проведен сравнительный анализ эффективности различных алгоритмов сортировки и перемножения матриц на разных данных с разной размерностью.